

QiMeng: Diseño Automatizado de Hardware y Software enfocado en procesadores

Diseño de alto Nivel - MP6160

Luis Chaves
Andy Corrales
Daniel Chacón
Andrés Sánchez

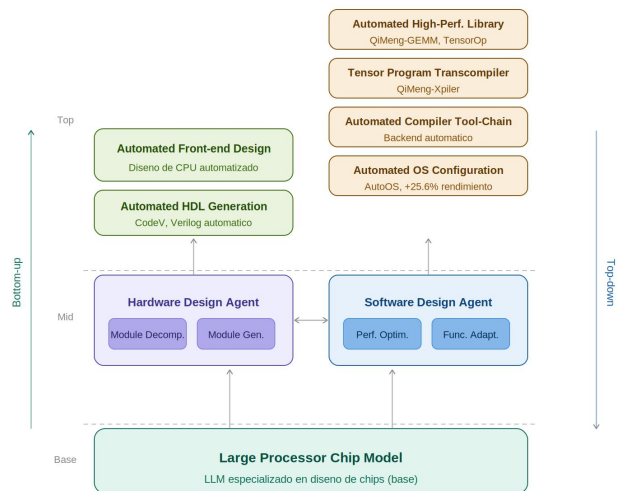
Problema a Resolver

- Desarrollar un modelo completamente automático, mediante el uso de la Inteligencia Artificial para el desarrollo de procesadores que involucren generación de hardware y software optimizados.
- Limitaciones de las metodologías actuales de diseño:
 - Restricciones en la tecnología de fabricación
 - Recursos limitados
 - Ecosistema diverso
- Debido a estas limitantes se requiere un nuevo paradigma de diseño que permita reducir costos, mayor eficiencia y rendimiento y que sea aplicable en aplicaciones diversas.

Metodología de diseño y arquitectura del LPCM

Fases:

1. Top-Down (actual)
2. Bottom-Up
3. Iteración



Por qué QiMeng necesita 3 capas?

Solución al problema de diseño automatizado

[1] Limite Fisico

Fabricacion sub-7nm alcanza efectos cuanticos. El rendimiento ya no escala con el proceso de fabricacion.

[2] Costo de Recursos

Ciclos de verificacion-iteracion requieren miles de horas-persona de expertos especializados.

[3] Diversidad de Ecosistemas

IA, edge computing y RISC-V exigen chips especializados y software a medida que los flujos actuales no cubren.

Top Layer - APLICACIONES

Diseño de chips: CPUs, compiladores, OS

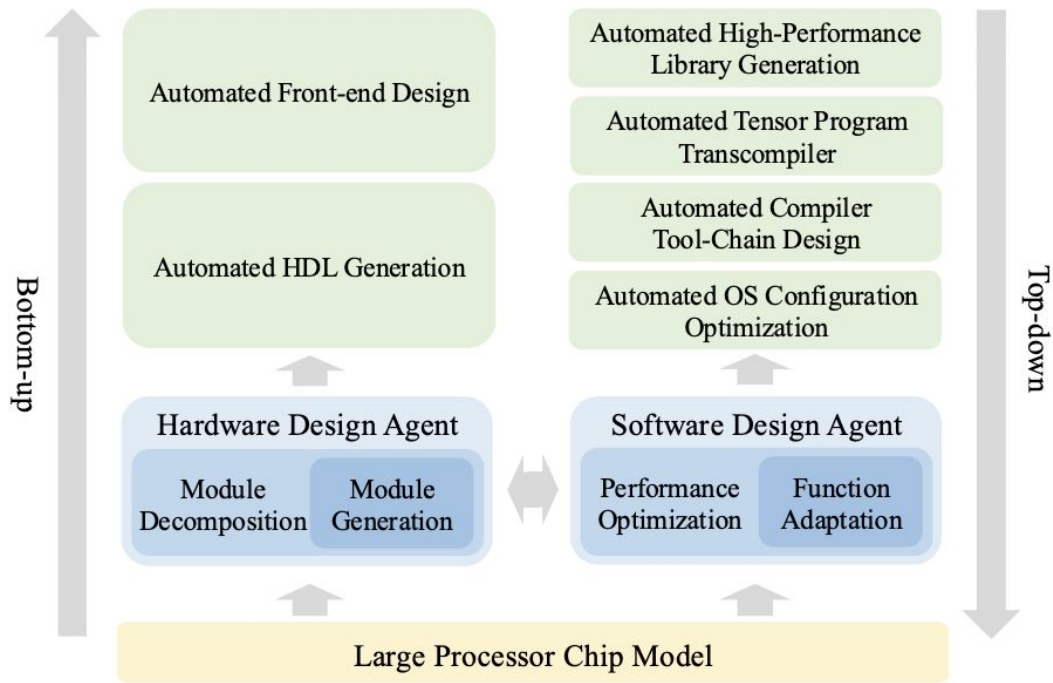
Mid Layer - AGENTES HW/SW

Hardware Design Agent + Software Design Agent

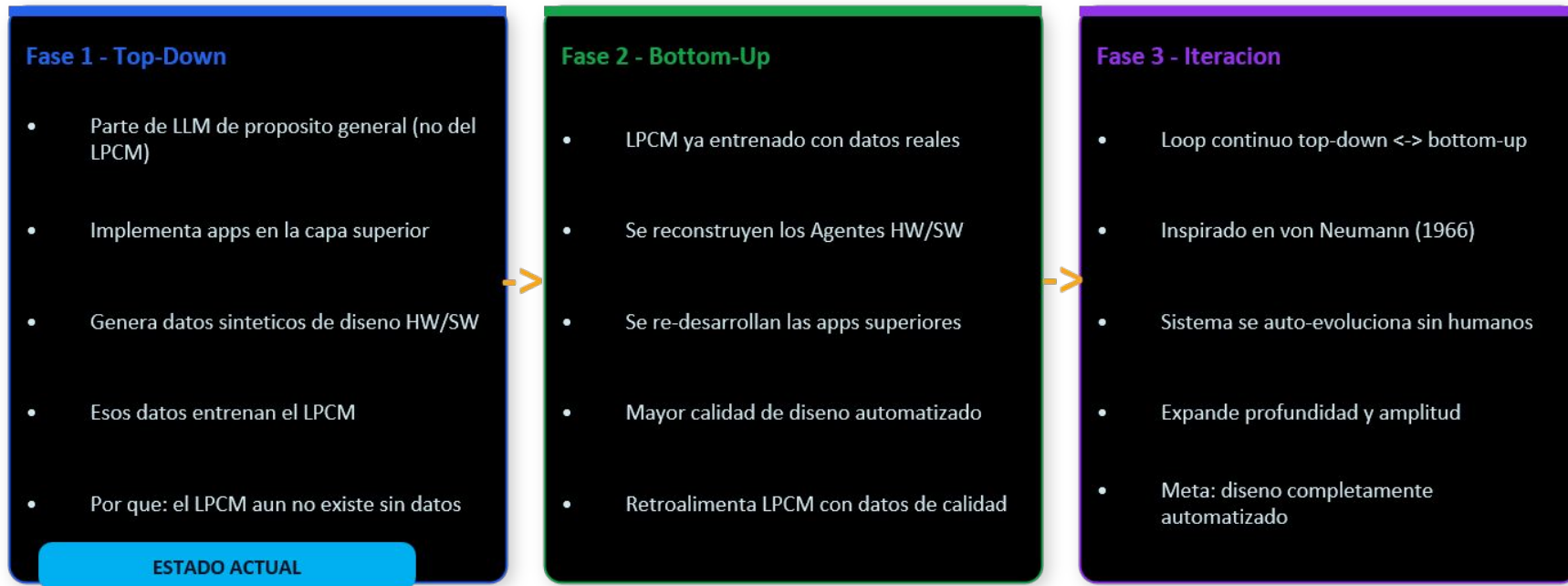
Base Layer - LPCM

Large Processor Chip Model (LLM especializado)

Diseño Bottom-up/Top-down



Fases del QiMing



Por que top-down es el unico inicio posible?

LPCM necesita datos de diseño, en Top-Down las apps generan los datos que necesita el LPCM. Posteriormente los agentes mejora, las apps también y se producen datos de mayor calidad

Diseño del LPCM

- **Paso 1 — Recolección por etapa:** Se recopilan datos separados para cada etapa del diseño de chips de forma independiente: bibliotecas HPC, OS, compiladores, diseño lógico, diseño de circuitos, diseño físico.
- **Paso 2 — Modelos por etapa y cascada:** Con esos datos, se entrena un modelo automatizado para cada etapa individualmente. Luego se **encadenan en cascada**: el modelo de la etapa superior genera una salida que sirve de entrada al modelo de la siguiente, y así sucesivamente
- Con la base de datos cross-stage ya construida, se entrena el LPCM usando 3 técnicas:
 - Chain-of-Thought (CoT) imitation learning
 - Curriculum learning
 - Reinforcement Learning

LPCM - Arquitectura Multimodal

ARQUITECTURA MULTIMODAL

Entrada (2 modalidades)

Texto + grafos (ASTs, DFGs, CFGs). GNN codifica topología -> contrastive learning alinea con texto -> embedding al LLM.

Salida (2 modalidades)

Código HW/SW + diagramas. Usa BSD y GRAPHARM para generación de grafos con topología correcta.

Por qué multimodal?

Los LLMs normales serializan grafos y pierden la topología. LPCM la preserva para capturar conocimiento real de diseño.

ENTRENAMIENTO CROSS-STAGE

El problema

Datos de diseño: escala terabyte vs petabytes del Internet. Datos cross-stage alineados son escasísimos.

La solución

Modelos por etapa entrenados -> cascada genera datos alineados multi-etapa automáticamente.

Técnica

CoT imitation learning + distribution alignment loss + curriculum learning (simple a complejo).

INFERENCIA DUAL-LOOP

Loop Externo (performance)

Árbol de búsqueda jerárquico. Poda de ramas de bajo rendimiento. Optimiza PPA del chip.

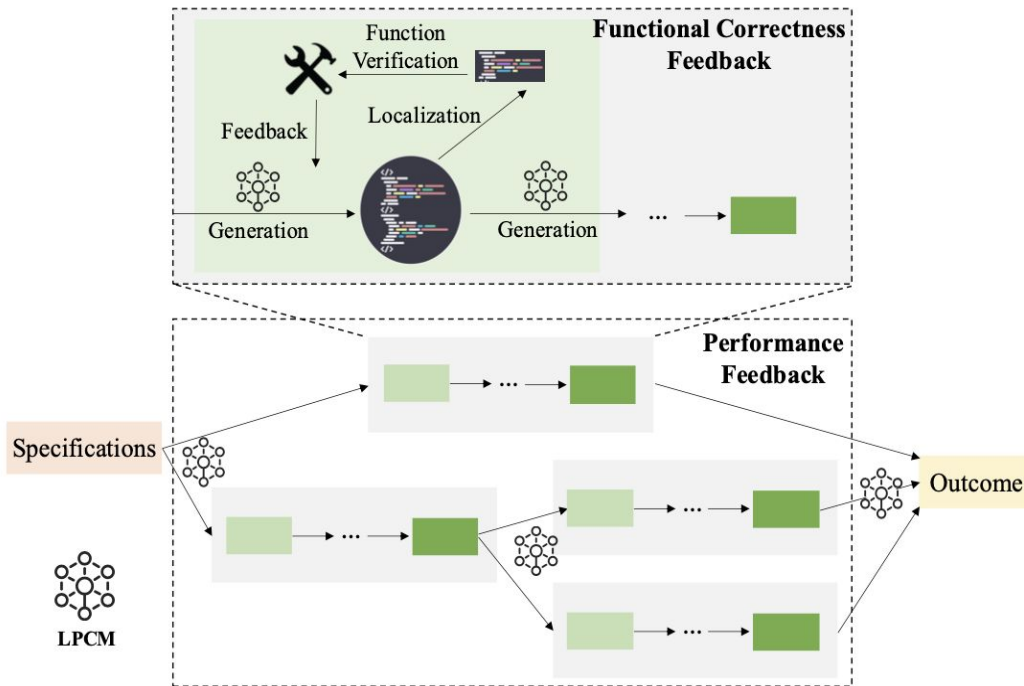
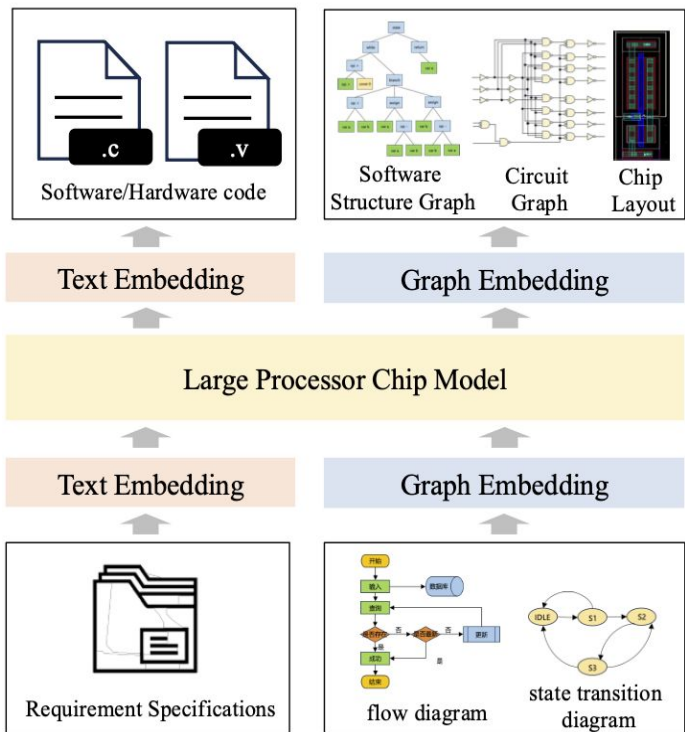
Loop Interno (correctness)

Verifica automáticamente, localiza errores, repara vía BSD + expansión de Shannon. Converge al 100%.

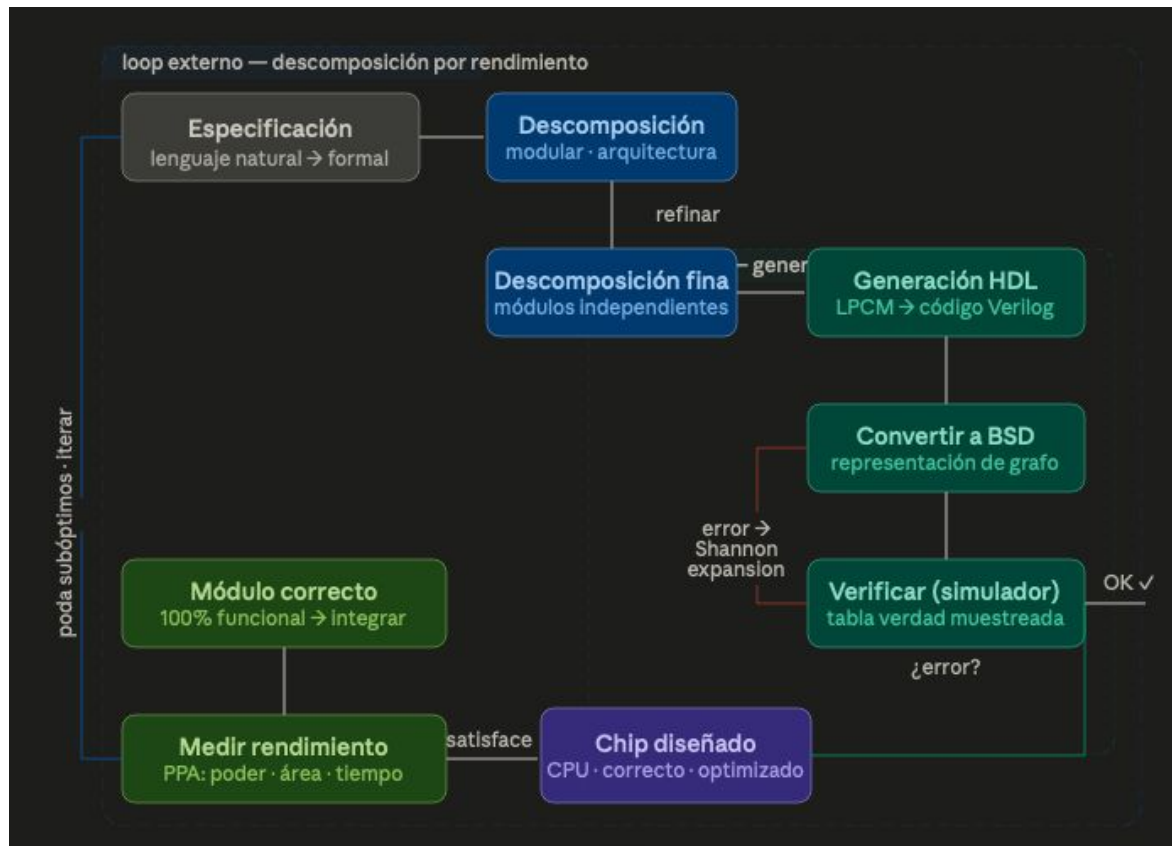
Resultado

El diseño es correcto y óptimo. Los dos loops operan de forma concurrente durante la inferencia.

Diseño/entrenamiento del LPCM



Hardware Design Agent



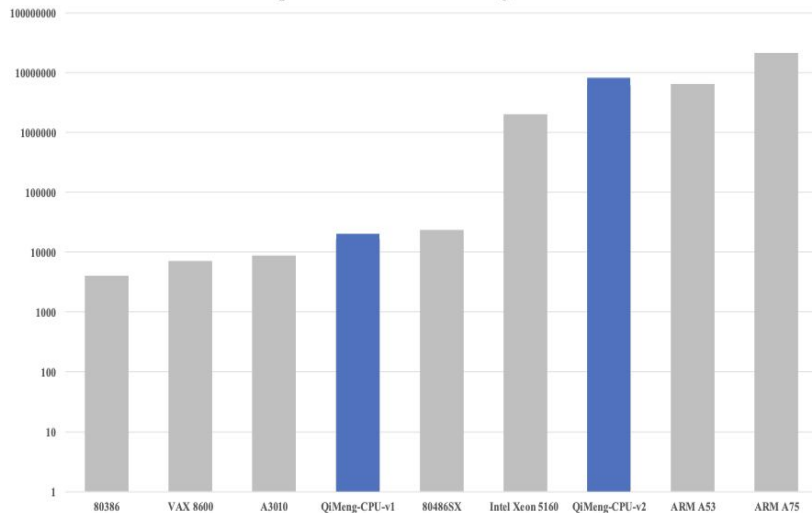
Hardware Design Agent ejemplo real

TABLE I

RESULTS OF AUTOMATED FRONT-END DESIGN BY QIMENG-CPU SERIES
COMPARED WITH EXISTING METHODS.

Target Circuit	Methods	Scale	Performance
Adder [29]	RL	118	NA
Circuit Modules [30]	DT	186	NA
Circuit Modules [31]	EL	~ 2500	NA
8-bit CPU [32]	LLM	999	NA
QiMeng-CPU-v1 (RISC V-32 CPU) [16]	BSD	~ 4 Million	1.62×10^4
QiMeng-CPU-v2 (Superscalar CPU) [33]	S-BSD	~17 Million	6.29×10^6

Performance comparison with human-designed commercial CPUs



CodeV - generación de HDL

TABLE II
COMPARISON OF OUR CODEV SERIES AGAINST VARIOUS BASELINE MODELS. RESULTS ARE CITED FROM THE ORIGINAL PAPER.

Type	Model	Model size	Open source	VerilogEval-Machine (%)			VerilogEval-Human (%)			RTL v1.1 (%)	
				pass@1	pass@5	pass@10	pass@1	pass@5	pass@10	Syntax	Func.
Base LLMs	GPT-3.5	-	×	60.9	75.0	79.9	33.5	45.9	50.0	79.3	51.7
	GPT-4	-	×	60.0	70.6	73.5	43.5	55.8	58.9	100.0	65.5
	StarCoder [35]	15B	✓	46.8	54.5	59.6	18.1	26.1	30.4	93.1	27.6
	CodeLlama [36]	7B	✓	43.1	47.1	47.7	18.2	22.7	24.3	86.2	31.0
	DeepSeek-Coder [37]	6.7B	✓	52.2	55.4	56.8	30.2	33.9	34.9	93.1	44.8
	CodeQwen [38]	7B	✓	46.5	54.9	56.4	22.5	26.1	28.0	86.2	41.4
	Qwen2.5-Coder [39]	7B	✓	66.2	79.2	83.9	34.6	45.6	51.0	89.6	41.4
Fine-Tuned LLMs	ChipNeMo [40]	7B	×	43.4	-	-	22.4	-	-	-	-
	RTLCoder-Mistral [41]	7B	✓	62.5	72.2	76.6	36.7	45.5	49.2	<u>96.6</u>	48.3
	RTLCoder-DS [41]	6.7B	✓	61.2	76.5	81.8	41.6	50.1	53.4	93.1	48.3
	BetterV-CL [42]	7B	×	64.2	75.4	79.1	40.9	50.0	53.3	-	-
	BetterV-DS [42]	6.7B	×	67.8	79.1	84.0	45.9	53.3	57.6	-	-
	BetterV-CQ [42]	7B	×	68.1	79.4	84.5	46.1	53.7	58.2	-	-
	CraftRTL-CL [43]	7B	×	78.1	85.5	87.8	63.1	67.8	69.7	93.9	52.9
	CraftRTL-DS [43]	6.7B	×	77.8	85.5	88.1	65.4	70.0	72.1	84.3	58.8
CodeV-Verilog	CodeV-Verilog-CL	7B	✓	78.1	86.0	88.5	45.2	59.5	63.8	93.1	62.1
	CodeV-Verilog-DS	6.7B	✓	77.9	<u>88.6</u>	90.7	52.7	62.5	67.3	89.7	55.2
	CodeV-Verilog-CQ	7B	✓	77.6	88.2	90.7	53.2	65.1	68.5	93.1	55.2
	CodeV-Verilog-QC	7B	✓	<u>80.1</u>	87.9	90.5	59.2	65.8	69.1	<u>96.6</u>	51.7
CodeV-All	CodeV-All-CL	7B	✓	78.5	85.6	87.6	46.6	58.8	62.5	<u>96.6</u>	55.2
	CodeV-All-DS	6.7B	✓	79.8	86.0	86.7	53.0	63.3	67.2	<u>96.6</u>	51.7
	CodeV-All-CQ	7B	✓	79.9	88.3	<u>91.1</u>	54.1	65.1	68.6	<u>93.1</u>	58.6
	CodeV-All-QC	7B	✓	81.9	89.9	92.0	56.6	67.9	71.4	<u>96.6</u>	55.2
CodeV-R1	CodeV-R1-Distill	7B	✓	76.2	85.6	87.0	<u>65.7</u>	<u>76.8</u>	<u>79.7</u>	-	<u>75.8</u>
	CodeV-R1	7B	✓	76.5	84.1	85.7	69.9	79.3	81.7	-	86.1

El agente de software: doble loop

Loop EXTERNO — Rendimiento

- Abstrae la optimización como un problema de búsqueda.
- Búsqueda guiada por LLM con Monte Carlo Tree Search (MCTS).
- Ciclo "observar–podar–optimizar–evaluar": poda ramas ineficientes.

Loop INTERNO — Adaptación

- Abstrae la adaptación como "generación de programas".
- El LLM genera el esqueleto; la síntesis simbólica corrige el detalle.
- Verify-repair con solver SMT Z3 hasta lograr equivalencia funcional.

Cuatro logros concretos en software

Aplicación	Resultado	Qué hace
AutoOS	+25.6%	Configuración del kernel de Linux (~15.000 opciones) frente a la optimización manual de expertos.
Compilador neural	>99%	Traducción C → ensamblador; backends generados con +70% de exactitud.
QiMeng-Xpiler	95%	Transcompilador tensorial cross-chip: CUDA · HIP · BANG C · VNNI ('Write Once, Run Anywhere').
QiMeng-GEMM	~2×	Librerías de alto rendimiento: duplica el rendimiento de OpenBLAS en RISC-V.

Tradeoffs, herramientas y por qué importa

Tradeoffs de diseño

- Complejidad: se doma con descomposición jerárquica + poda.
- Escalabilidad: de miles a millones de compuertas (escala industrial).
- Flexibilidad/portabilidad a costa de un sistema con muchas piezas por integrar.

Herramientas clave

- MCTS y Tree of Thoughts (exploración del espacio).
- Solver SMT Z3 (síntesis y reparación).
- BSD / expansión de Shannon (hardware).
- Meta-prompts + Test-Time Scaling sobre los LLMs.

Relevancia actual

- Reduce el cuello de botella del HDL (+70% del ciclo, según NVIDIA).
 - Habilita ecosistemas de chips no-NVIDIA (RISC-V, AI chips domésticos).
- Democratiza el diseño: menos dependencia de expertos escasos

Promesa y realización parcial

Ventajas

- Primer enfoque integrado HW + SW totalmente automatizado.
- Logros de escala industrial: 1.ª CPU y 1.ª CPU superescalar automáticas.
- Resultados de SW medibles y modelos abiertos competitivos.

Limitaciones

- El sistema NO está completo: solo en la Fase 1 del roadmap.
- Las CPUs v1/v2 aún NO usan el LPCM.
- El Software Agent solo explota texto (grafos = trabajo futuro).
- No supera a librerías manuales en GPUs alta gama

Uso de IA

Se utilizó IA para la generación de las tablas en la diapositiva 4, 6 y 8. Así como para generar la imagen de la diapositiva 10, esto para explicar de una manera más resumida y más gráfica lo que se presenta en el paper. Así como para obtener una lista de cosas que debíamos mencionar en la presentación.

“Dado el contexto de este paper dame una tabla resumen de cómo se entrena la LPCM”

“Dado el contexto de este paper dame una imagen que explique con más detalle la parte del Hardware Design Agent”

“Dame una lista de cosas puntos que debemos mencionar sobre este paper si tenemos que hacer una presentación de 15 minutos”